

EXERGYNET

Infrastructure for Persistent, Governed and Economically Accountable Machine Intelligence

Model-independent state, execution-state mobility, and measured efficiency for AI systems that must persist across models, sessions, machines, and organizations.

Seven Ezumba, Chief Architect

August 2026

Abstract

Artificial intelligence is becoming more capable and more substitutable at the model layer at the same time: new models arrive quarterly, model choice is increasingly a commodity decision, and the cost of a single inference call continues to fall. What has not fallen is the cost of everything a persistent, multi-step, multi-agent system must do around that inference call — reconstructing state, re-fetching evidence, coordinating multiple models and tools, and producing a record of what actually happened. That cost is paid in compute, memory, storage, network, energy, and engineering effort, and it does not disappear just because the model got better.

That gap motivates ExergyNet’s first and most concrete question, which is economic before it is architectural: **can a system reduce the resource cost required to complete a validated useful task, measured against a workload’s own existing, optimized baseline — without requiring that workload to abandon its chosen model or infrastructure?** On a single NVIDIA H200 with a Nemotron-class model, ExergyNet’s xLMP memory layer held staged prompt cost roughly flat at ~660–820 tokens as the underlying corpus grew from 8,000 to 285,000 tokens, delivered a 24.4-point accuracy advantage over a tested RAG baseline at equal evidence budget, and reached roughly 11.3x the correct-task throughput of full-context replay at each method’s best compliant load. These are bounded, tested-envelope results, not a universal savings claim, and they are reported that way throughout this paper.

Efficiency alone is not the product. The durable requirement underneath it is different in kind: **models can change. Institutional state must persist.** This paper reports a second, newer result in that same spirit, at a different layer: within a frozen same-model runtime envelope, specific components of a model’s own live execution state were isolated and shown to move across physically distinct machines — different CPU vendors, different hosts — and independently reproduce the source machine’s continuation behavior. This does not establish universal execution-state portability, and this paper is explicit about what it does and does not show; it does establish that execution state, like persistent memory, is decomposable into named, independently testable components, some of which travel.

ExergyNet is the architecture built to hold both requirements together: a model-independent substrate that separates reasoning from authority, memory from authorization, and economic allocation from consequence control, organized into three coordination planes covering persistent state, coordination and authority, and resource accounting. A fourth idea threads through the paper’s newest section: **state may move without authority moving with it** — the same separation this paper already applies to memory and money, extended to a model’s own live execution state.

This paper reports that architecture, its measured results within their tested envelopes, and the

research frontier that remains open — including where cross-model and cross-runtime execution-state portability is not yet attempted, where the mechanism behind the results in Section 5 is not yet known, and where policy enforcement runs today in a monitored pilot rather than as a system’s sole authority.

As machine intelligence becomes economically productive at industry scale — a scale now measured in tens of billions of dollars of quarterly infrastructure investment by the industry’s largest supplier — the infrastructure connecting independent, heterogeneous intelligences to shared state, evidence, and consequential action becomes as consequential as the models themselves.

Executive Summary

The economic starting point. Model inference is getting cheaper and more interchangeable. The resource cost of persistent, state-heavy, multi-agent work is not falling at the same rate, because that cost lives in reconstruction, re-fetching, coordination, and verification — not in any single model call. This is the first question a buyer should ask about ExergyNet: does it reduce that cost, measured against the workload’s own baseline, without requiring a change of model or cloud?

What ExergyNet does not require. See the box immediately below — the short answer is: no required model change, no required cloud migration, no required exclusive vendor, no transfer of policy authority to ExergyNet, and no consequential permission that ExergyNet originates on its own.

The architecture. Once the economic case is established, ExergyNet’s deeper product becomes visible: customer-controlled, model-independent authoritative state, together with a live execution-state layer that is a distinct object from that persistent state. ExergyNet sits at the boundary between a reasoning engine and the digital or physical systems it can affect (**Figure 1**), organized into three coordination planes (**Figure 2**): Knowledge & State, Coordination & Authority, and Resource & Economic Coordination. As models and agents multiply, that same state becomes the basis for interoperability across organizational boundaries; where actions become consequential, external authority and provenance become necessary. The organizing principle throughout is separation: reasoning and authority are never the same mechanism, and, as Section 6 and the new Section 7 make explicit, neither are execution state and the authority to act on it.

Five headline results, in the order a buyer should weigh them:

Result	Measured outcome	Tested envelope
Useful-work / state efficiency	Prompt cost held ~660-820 tokens flat as corpus grew 8k-285k; full-context grew to 67k and was rejected past 262k	H200, synthetic corpus
State governance	False authoritative commitments fell 8% to 0%; model accuracy held at 84%	100 real procurement cases
Deterministic authority	Rejected a live prompt-injection after the reasoning model complied; reproduced twice	Live listener, unit-tested

Result	Measured outcome	Tested envelope
Execution-state mobility	Same-model cross-host recurrent-state realization demonstrated; a two-block region of one state component was each independently sufficient to reproduce the source host’s trajectory, and a second component was reduced to a two-block interacting pair within the tested search	Frozen same-model runtime, one workload length, two physical x86-64 hosts
Physical consequence	False-release held at 0/50 in two independent runs vs. 20-34% ungoverned	Sealed synthetic holdout

The larger implication. As machine output becomes economically productive at industry scale, the resources and coordination mechanisms that sustain it become strategic infrastructure in their own right, not implementation detail.

What ExergyNet Does Not Require

ExergyNet does not inherently require an adopting organization to:

- replace its frontier model or reasoning engine of choice;
- adopt one exclusive model vendor;
- move all enterprise data into an ExergyNet-owned store;
- transfer policy authority to ExergyNet; or
- allow ExergyNet to originate consequential permissions on its own.

Each of these is a property of the architecture described in this paper, not a marketing assertion: Section 3 shows the model as an interchangeable component at the boundary, not a replaced one; Section 4 introduces the Context/Memory/Authoritative-State distinction that keeps a customer’s own data as the thing being preserved, not relocated; Section 7 states plainly that authority must originate externally from the institution using the protocol, and that this holds even for a model’s own execution state. **Exception, disclosed:** where an organization chooses to route settlement through ExergyNet’s own test-network infrastructure (Section 10), that specific, opt-in function does involve an ExergyNet-operated component; it is not a requirement of adopting the memory, state, or authority layers described elsewhere in this paper.

1. The Economic Problem

Every independent reasoning system begins each session from nothing durable: state is reconstructed at full cost, evidence is re-fetched without stable identity, authority is implicit in whatever credentials the caller holds, and no independent record survives to explain why an action occurred. This is a property of the surrounding architecture, not of any particular model, and improving the model does not change it. The cost is not paid once — it is paid every session, every model swap, and every hand-off between agents, in compute, memory, storage, network, energy, and engineering time.

This argument is no longer only architectural — it is now visible in the infrastructure economics of the companies building frontier compute. In its most recent quarter, NVIDIA reported \$89.0 billion in Data Center revenue, up 117% year-over-year, and supply and capacity purchase commitments that rose from \$119 billion to \$279 billion quarter-over-quarter — an increase its finance leadership attributed primarily to the procurement of memory. At the same time, independent technical disclosures from memory manufacturer Micron reported that compute performance is scaling roughly 3x every two years while high-bandwidth memory (HBM) capacity is scaling less than 2x over the same period (**Figure 9**).

These figures are external, dated evidence about the industry, not ExergyNet results, and they do not validate this architecture. What they confirm is the premise this paper derives independently: memory, compute, energy, and physical infrastructure have become contended, strategically managed resources — exactly the resource layer that a workload’s own cost-per-validated-task must eventually measure, allocate, and account for (Sections 2 and 10).

The question this paper answers first is therefore economic, not architectural: given a workload an organization already runs, on infrastructure it already operates, can the resource cost of completing a validated useful task go down — measured against that workload’s own existing, optimized baseline — without requiring a different model or a different cloud? Section 2 makes that question precise. Section 4 reports the measured result. Only after that does this paper introduce the architecture that makes the result durable rather than a one-time optimization.

2. Useful Work: A Measurement Layer for Machine Intelligence

Before describing the architecture, this paper defines the commercial measurement problem it is built to answer. Raw token count is not the right final denominator — a workload that halves its tokens but doubles its retries or triples its coordination overhead has not become more efficient. The correct denominator is the resource cost of a **qualified successful task**: one whose output meets whatever correctness or validation bar the workload itself already requires.

Define the cost per qualified successful task as:

$$C_Q = \frac{C_{\text{total}}}{N_{\text{qualified successful tasks}}}$$

where total cost includes, where measurable:

$$C_{\text{total}} = C_{\text{inference}} + C_{\text{retrieval}} + C_{\text{state}} + C_{\text{verification}} + C_{\text{retries}} + C_{\text{coordination}} + C_{\text{network}} + C_{\text{storage}}$$

and, symmetrically, define useful-work efficiency as the inverse ratio against total energy consumed:

$$E_Q = \frac{N_{\text{qualified successful tasks}}}{E_{\text{total}}}$$

alongside the completion-time distributions T_{p50} and T_{p95} , since a workload’s tail latency is often as commercially relevant as its mean cost.

Currently measured. The results reported in Section 4 measure specific terms of C_{total} directly — principally $C_{\text{inference}}$ and C_{state} , via staged prompt-token count as a cost proxy on a fixed hardware

target (a single NVIDIA H200). They do not yet measure C_{retries} , $C_{\text{coordination}}$, C_{network} , or C_{storage} as independent, itemized terms, and E_Q is not separately measured in this paper — energy is not directly instrumented in the reported results.

Proposed standardized evaluation framework. C_Q , E_Q , and the full multi-term cost decomposition above are presented here as the target evaluation framework for the benchmark program this paper’s research frontier describes (Section 13), not as an existing, fully-instrumented result. ExergyNet has not measured every term in C_{total} for any reported workload, and this paper does not claim otherwise. The framework itself is architectural and research framing: a standard against which future, more complete measurements can be reported and compared, workload by workload, against each workload’s own optimized baseline — never as a universal efficiency multiplier.

3. The ExergyNet Architecture

ExergyNet occupies the boundary between a reasoning engine and the world it can affect. The reasoning engine is interchangeable — GPT, Claude, Nemotron, an open-weight model, a robotics controller, or a human operator — and the architecture is drawn accordingly: models and applications connect through ExergyNet’s protocol components to state the customer owns, inside a boundary the customer controls (**Figure 1**). ExergyNet does not hold that state on the customer’s behalf as a precondition of using it, and it is not drawn as a box every request must pass through with no visibility into what happens inside.

The architecture organizes into three coordination planes (**Figure 2**):

- **Plane I — Knowledge & State.** What exists, what happened, and what is currently authoritative: xLMP / Exergy Vault, deterministic indexing and routing, Temporal Authority, and physical observation (Edge Witness).
- **Plane II — Coordination & Authority.** Who may interact with what, under what constraints: machine identity, the xISA capability vocabulary, Vanguard model routing, the LNES-22 Consequence Boundary, and AERIS external-evidence acquisition.
- **Plane III — Resource & Economic Coordination.** What was consumed, authorized, and settled: RHO resource accounting, Omega, and MMS settlement.

The architecture keeps a small set of predicates permanently separate: memory evidence is not execution authority; model identity is not agent identity; agent identity is not economic authority; economic authority is not consequence authority; and resource accounting is not market value. Section 7 extends this same discipline to a model’s own execution state. These separations are what let intelligence be swapped, fail, or be distrusted without the surrounding system losing what it knows, what it may do, or what it must account for.

Security posture. ExergyNet assumes the model may be compromised. Consequential authorization is placed outside the reasoning model so that protected actions can be evaluated deterministically, independent of whether the model that proposed them can be trusted (Section 9).

4. Persistent State and xLMP

The clearest cost of statelessness is reconstruction: an agent that rebuilds its world every time it changes model, machine, or session is, in effect, rebuilding the road behind itself every time it moves. xLMP is ExergyNet’s answer — a root-addressed, content-verified persistent evidence store with bounded retrieval, and the paper’s primary measured answer to the economic question posed in Sections 1 and 2.

On a single NVIDIA H200 with a Nemotron-class model over a synthetic corpus, xLMP held staged prompt cost roughly flat at ~660-820 tokens as the underlying corpus grew from 8,000 to 285,000 tokens, while full-context injection grew to 67,000 tokens and was rejected outright past 262,000 (**Figure 4**). At equal evidence budget, xLMP retrieval reached 92.4% accuracy against a 68.0% tested RAG baseline — a 24.4-point advantage — and delivered roughly 11.3x the correct-task throughput of full-context at each method’s best compliant load (**Figure 5**).

Why this is a capacity result, not only a cost result. Reducing the context staged into active working memory per operation means the same physical hardware sustains more useful machine work per unit of memory and time. Under constrained memory and compute, that is a capacity property, not merely a savings — and it is precisely the C_{state} term of the useful-work cost decomposition introduced in Section 2.

Validation scope. These results hold within the tested H200 / synthetic-corpus envelope; broader-scale validation is part of the ongoing measurement program described in Section 13. A separate efficiency figure (~42.7x vs. full-context, ~4.0x vs. RAG) is reported pending independent re-verification of its source artifact.

VMN provides a local, open-source implementation of xLMP’s persistent-state model — a developer-facing node that runs without a cloud dependency. VMN’s own local operation is independent of the execution-state research reported in Section 5; integrating VMN’s persistent project state with the portable execution-state package that Section 5 and Section 7 describe is itself part of this paper’s stated research frontier (Section 13), not a completed integration.

Context, Memory, and Authoritative State

Efficiency mechanisms — caching, compression, retrieval — reduce what a model is shown for a given task. They do not, by themselves, answer a different question: what is the institution’s current, canonical record of what is true, and who is allowed to change it? This paper draws a firm line between three concepts that are commonly blurred:

- **Context.** Information supplied to the reasoning engine for the current inference. Exists only for the duration of that call.
- **Memory.** Information retained because it may be useful in later reasoning. Improves recall; does not, by itself, establish what is canonical.
- **Authoritative State.** The externally maintained representation of what the system currently treats as canonical, including, where applicable: version, source, provenance, authority, evidence, permissions, and transition history.

Context $\neq$ Memory $\neq$ Authoritative State

This distinction is commercially load-bearing, not academic: provider-native prompt caching and memory systems are rapidly commoditizing the first two categories. A vendor that reduces token cost or improves recall has not, by that fact alone, produced a system that knows what is currently true, who said so, and under what permission it may be acted on. Section 5 introduces a fourth, distinct concept this paper does not conflate with any of the three above: a model’s own **live execution state** — the runtime-resident representation a reasoning engine is actively using mid-computation, as opposed to anything retrieved into context or persisted as memory or authoritative state. This paper does not make claims here about any specific competing system’s architecture; the distinctions

are stated on their own terms, and the reader can apply them to whatever alternative is under evaluation.

5. Execution-State Mobility

5.1 Why execution state is distinct from persistent memory

Section 4 distinguishes context, memory, and authoritative state. A fourth object is distinct from all three: the live, runtime-resident execution state a model is actively using mid-computation — for a hybrid attention/recurrent architecture, this includes both attention key-value cache and recurrent-layer state (convolutional and state-space components). Execution state is not evidence retrieved into context, not information retained for later recall, and not an institution’s authoritative record — it is the model’s own momentary computational condition. Whether and how that condition can be captured, moved, and re-realized on a different physical machine is an empirical question this section reports on directly, separate from anything Section 4 measures.

5.2 Same-process and fresh-process controls

On a hybrid KV/SSM execution runtime, same-process erase-and-restore reproduced continuation exactly across every tested context length (30 of 30 aggregate trials), with bit-perfect capsule round-trips (9 of 9 checked). An initial characterization of fresh-process restoration — a new process loading a previously saved state — found exact reproduction for short contexts (10 of 10) but divergence for medium and long contexts in the tested runs, isolating a first research finding: **serialized-state equality is not the same as live-execution-state equality**. That original medium/long characterization was later found to conflate two distinct effects, corrected in the following section.

5.3 Runtime-state realization and checkpoint metadata

A subsequent audit isolated the reason for the original medium/long divergence to cross-workload checkpoint contamination in the test harness, not to a general failure of fresh-process restoration. Once corrected, fresh-process behavior was re-characterized per workload independently, and a distinct efficiency question was separated from the underlying correctness question: a saved checkpoint’s *core capsule* correctness (does the restored state decode to the right content at all) is not the same property as its *first-use* efficiency (how many tokens a freshly-restored process must re-process before it is fully warm). Measured across the corrected harness, restoration required partial re-evaluation of a small, fixed number of tokens (four, in the tested configuration) before reaching steady state — a first-use cost distinct from, and smaller than, full re-prefill.

5.4 Physical-host transfer

A same-model, frozen-runtime state-mobility protocol was defined, version-controlled, and frozen before any cross-host trial was executed, specifying the exact restore sequence, runtime profile, generation request, and success predicate in advance. Two physically distinct x86-64 hosts were used: one built around an AMD EPYC 74F3 processor with GPU-accelerated inference, the other around an Intel Xeon Platinum 8573C processor with CPU-only inference — the same model file (verified by SHA-256 hash) and the same frozen llama.cpp-family runtime binaries (also hash-verified identical across both hosts) on both.

An initial canary campaign transferring a complete saved-state package from the source host to the destination host reproduced the source host’s exact token-level continuation for the short-context workload, but not for medium- or long-context workloads. Root-cause investigation first isolated the divergence to the broader host execution environment rather than to any specific component of the transferred state — including a control condition in which the destination host, given only the source host’s prompt with no restored state at all, still diverged from the source host on medium and long workloads. That control result means whole-package cross-host divergence, by itself, does not establish that transferred execution state was the cause; it could equally be explained by ordinary host-to-host execution differences that would appear regardless of any transfer. Isolating what, specifically, is and is not portable required decomposing the transferred state into its named components, reported next.

5.5 Recurrent-state decomposition

The model’s recurrent-layer sidecar state decomposes into two distinct tensor families per layer: a convolutional component ($\mathbf{R}$, $\mathbf{R_CONV}$) and a state-space component ($\mathbf{S}$, $\mathbf{S_SSM}$) — together, not attention key-value cache, which this architecture’s hybrid design keeps separate. Transplanting the source host’s $\mathbf{R_CONV}$ component alone onto the destination host’s otherwise-native state was independently sufficient to reproduce the source host’s reference trajectory for the tested medium-length workload; transplanting the source host’s $\mathbf{S_SSM}$ component alone was independently sufficient as well. This is the first result in this section establishing that a specific, named slice of live execution state — not the whole undifferentiated package — can be transferred across physically distinct hosts and correctly re-realized.

5.6 S-state alternative sufficiency

Having established that $\mathbf{S_SSM}$ state is transferable, an adaptive search across the model’s recurrent layers (block-ordinal search, hybrid bisection and targeted single-block trials) narrowed the region carrying this effect to the model’s first eleven recurrent-layer positions, then found that transplanting the $\mathbf{S_SSM}$ state at model block 9 alone from the source host was sufficient to reproduce the source-host trajectory, and, independently, that block 11 alone was also sufficient. A separate necessity test then removed each of these blocks individually from the full eleven-block positive region: removing block 9 alone left the region still sufficient, and removing block 11 alone left the region still sufficient. Neither block is therefore individually required for the effect within the tested background — multiple alternative sufficient sub-configurations exist within the same tested region.

5.7 R-state pair interaction

The $\mathbf{R_CONV}$ component’s positive region did not decompose the same way: splitting the eleven-block positive region in half made both halves independently insufficient, indicating the effect there depends on an interaction between blocks rather than any single block. An adaptive delta-debugging search — a systematic method for shrinking a set of contributing factors to one where removing any single remaining member breaks the effect — reduced the positive region to a two-block pair, model blocks 4 and 21, within the tested search. Both blocks were confirmed necessary: removing either one from the pair alone was insufficient, and no alternative two-block or larger route to the same effect was found within the trials run. This is reported as a **1-minimal sufficient set within the tested search space** — a set from which no single member can be removed without losing sufficiency — not as the globally smallest possible set, since the full space of possible block combinations was not exhaustively searched.

5.8 What was demonstrated

Within a frozen same-model, same-runtime, same-quantization envelope, on one tested medium-length workload, across two physically distinct x86-64 CPU platforms (an AMD EPYC and an Intel Xeon host): a model’s live recurrent execution state decomposes into independently testable named components; specific components of that state, transplanted alone, are sufficient to reproduce the source host’s reference continuation on the destination host; one component’s effect is carried by either of two individually-sufficient, mutually non-required blocks; the other component’s effect requires a specific two-block pair with no individually-sufficient member and no alternative pair found within the tested search. This is a same-model, cross-host result about execution-state structure and mobility. It is not a claim that any of this generalizes across models, runtimes, quantizations, workload lengths, or host counts beyond what was tested.

5.9 What remains open

The mechanism producing the recurrent-state cross-host divergence in the first place — the reason execution differs between an AMD EPYC and an Intel Xeon host at all, given identical model weights and identical application binaries — is not established by this work and is reported as unknown, not as a settled numerical-precision or CPU-vendor explanation. Also open: whether the same decomposition and the same specific blocks reproduce on a third physical host; whether the same structure holds at short or long context lengths, under a different quantization, on a different runtime, or on a different model family; whether the `S_SSM` region’s minimal structure (beyond the two blocks already shown alternative-sufficient) can be reduced further; and how a portable execution-state package of this kind should be integrated with the persistent, authoritative project state described in Section 4 — addressed at the architecture level, not yet the implementation level, in Section 7.

6. The Model-Substitution Principle

A model-independent substrate is only as valuable as its answer to a specific, concrete event: a model is replaced. Not hypothetically — routinely, as pricing, capability, and vendor relationships change.

$$M_A \rightarrow M_B \rightarrow M_C$$

while $S_{\text{institution}}$ must remain continuous.

Models can become interchangeable without institutional truth becoming disposable.

Section 5’s results sharpen exactly what this claim does and does not rest on. The execution-state mobility reported there is a **same-model** result: a model’s live execution state was shown to move across physically distinct hosts running the identical model, not across different models. It does not establish, and this paper does not claim, that execution state moves across a *model substitution* the way this section’s principle requires. What Section 5 does strengthen is a narrower, still load-bearing point: execution state is decomposable and, within a frozen model, portable — which matters for infrastructure engineering (Section 7) even though it is a different claim from model-independent portability.

This paper therefore distinguishes two portability classes rather than treating “state” as one thing:

	Portability class	Status
Persistent / authoritative project state (Section 4)	Model-independent by architecture — defined and maintained independently of which model is reasoning over it	Architectural property, demonstrated within the tested envelopes reported in Sections 4 and 8
Model-native live execution state (Section 5)	Currently validated only within a same-model, frozen-runtime envelope	Cross-model execution-state movement is not demonstrated and is not claimed

The infrastructure can be model-independent even when a model’s native execution state is not. That is the precise, bounded form of the model-substitution principle this paper defends: the authoritative state described in Section 4’s boxed distinction is defined independently of any one model’s internal representation, which is what makes substitution a coherent target regardless of what is or is not yet known about moving a model’s own live execution state across a model change. The point of this section is the requirement the architecture is built to satisfy, not a claim that the hardest form of it — silent, lossless state continuity across a full model substitution — has already been demonstrated.

7. Portable Intelligence Packaging

Status: defined / specification. This section describes an architectural pattern this paper is proposing based on the boundaries Sections 4–6 establish; it is not describing a shipped implementation, and no claim in this section should be read as a current-production status claim.

Sections 4 and 5 establish two different kinds of state with two different portability profiles: persistent, authoritative project state that is architecturally model-independent, and live execution state that this paper has shown, within a tested envelope, to be decomposable and partially portable across physical hosts running the same model. A portable machine-intelligence package — the unit that would need to move if an agent’s work is to continue on a different machine, under a different model, or under a different institution’s infrastructure — has to keep these and several other properties separate rather than collapsed into one artifact:

- **Project identity** — which piece of work this is, independent of who is executing it.
- **Model identity** — which reasoning engine produced or is producing the work.
- **Runtime identity** — which specific binary and execution environment realized that model.
- **Native execution state** — the model-and-runtime-specific live state Section 5 studies.
- **Persistent project state** — the model-independent record described in Section 4 (implemented locally today via VMN/xLMP).
- **Evidence and provenance** — what is known and on whose authority.
- **Capabilities** — what the package is technically able to request.
- **Economic authority** — what spending has been granted to it.
- **Consequence authority** — what real-world or system actions it may trigger.
- **Compatibility requirements** — what model, runtime, and quantization the native execution state, if any is carried, actually requires to be re-realized.

The governing invariant is the same one this paper already applies to memory and to money, extended to execution state itself:

Execution state may move, and Section 5 reports a tested case where a specific slice of it did. Authority — economic or consequential — does not move with it, and must be re-established at the destination under that destination’s own authority mechanisms (Sections 8–9 for the general case; LNES-22’s Consequence Boundary for the enforcement mechanism). Portable Intelligence Packaging is the architectural name for keeping these properties separate as this paper’s execution-state, memory, and authority work continue to develop together; it is a specification this paper is stating precisely so that future implementation work can be checked against it, not a system that exists today.

8. Administrative Boundaries and Interoperability

Some of the strongest requirements this architecture answers only appear once more than one administrator is involved. Consider a scenario with no single trusted administrator: a company evaluating a vendor’s model output, a government agency requiring an audit trail independent of the vendor, a model provider that cannot see the customer’s internal data, and a cloud provider that hosts infrastructure for both without adjudicating disputes between them. None of these parties can be asked to simply trust another’s internal state.

(Figure 3B.) Two organizations, each retaining their own internal systems and their own authoritative state, are bridged only at the interaction boundary — not merged, not placed under one administrator, and not required to expose their internal systems to each other or to ExergyNet beyond what the interaction itself requires. This is the stronger version of a “neutral ground” position: not an absolute claim about corporate neutrality, but a property demonstrated through the architecture itself — verifiable state and evidence exchanged at the boundary, nothing more.

The integration-cost argument, correctly bounded. As the number of independent, heterogeneous intelligences and organizations (N) in a shared workflow grows, unconstrained pairwise integration between every pair carries a potential quadratic cost; a shared substrate instead lets each participant implement one common interface **(Figure 3A)**. This is a potential integration bound, not a measurement of actual network traffic in any deployed system, and it does not imply that every model needs ExergyNet. The defensible conclusion is narrower and remains true even in a world where most participants never adopt this protocol: **if independently governed systems require repeated bilateral state, identity, evidence, or authority integrations, common protocol semantics can reduce the bespoke integration burden those systems would otherwise each pay separately.**

9. Reasoning, Evidence, and Authority

Reasoning and authorization are architecturally separate. A proposed action must independently clear four gates — identity, capability, freshness, and integrity — before it is treated as authorized; failing any one gate is a denial **(Figure 7)**.

The strongest result. ExergyNet sent a live prompt-injection payload to a running authority-review listener. The reasoning model partially complied: it returned an approval and did not flag the injection despite its own instructions to do so. The deterministic validation layer rejected the request outright before any signed review could be produced — no credential was exposed at any point, because key material is structurally excluded from what reaches the model. The attack was

reproduced after a logging fix, with the same rejection now producing a durable, signed record. The finding, bounded to this validated run and stated as evidence for one specific architectural principle — **reasoning and authority must remain separable, not that ExergyNet should govern every model** — is this: a reasoning model can fail adversarially without the authority system failing with it.

Supporting validation: 64 assertions across three independent unit-test scripts; 38 of 39 adversarial tests passed against the agent-authorization API (the one failure was a test-environment artifact); and 100 of 100 concurrent double-allocation attempts against a shared balance were correctly serialized.

Evidence integrity is separately governed. State-governance testing over 100 real procurement-decision cases reduced false authoritative-state commitments from 8% to 0% while model judgment accuracy held flat at 84% (**Figure 6**) — the substrate prevented unresolved evidence from being promoted into an authoritative record, within the tested case set. This distinction matters commercially: **the model did not become more accurate; the surrounding system prevented unresolved evidence from becoming authoritative state.** Better governance is not the same thing as better model intelligence, and this result is evidence for the state-centric product definition in Section 4, not for the model’s own reasoning quality.

Validation scope. The identity/capability/freshness/integrity validator is live and independently tested. The broader policy-capability gate is implemented and tested but currently runs in a monitored pilot mode: it evaluates and logs every real decision without yet being the sole authority over execution (Section 13).

10. Machine Resource Economics

If machine output is economically productive, the resources consumed to produce it — compute, memory, storage, network, energy, and time — become economically significant in their own right. ExergyNet’s answer is a resource-authority pipeline: humans fund an agent identity, which receives bounded economic authority, spends it under explicit policy, and produces an auditable settlement record (**Figure 11**). As Section 7 makes explicit for execution state generally, an agent identity or a granted budget intent is not, by itself, spending authority — economic authority is a separate predicate that must be independently established, not inferred from identity or intent alone.

RHO is ExergyNet’s resource-cost accounting unit, defined by measurement, not by market price. A metrology pass fixed a measured cost basis across two hardware classes; the operations with the lowest cost are measured directly at the host, while the highest-cost operation (dominated by GPU inference) is currently modeled from pricing data rather than measured on-device — the next stage of the metrology program. As a concrete reference point, the frozen G0 accounting basis defines 1 RHO_G0 as corresponding to \$0.0001 of reference ExergyNet economic-capacity cost — an accounting normalization, not a market price, a stablecoin peg, a USD redemption promise, a physical energy-unit definition, or a token price. RHO’s value is defined independently of any market price, token, or energy interpretation, and this paper does not imply otherwise. Separately, any metered or shadow-tariff pricing work referenced elsewhere in ExergyNet’s internal documentation is explicitly a shadow policy candidate, not the production settlement policy this paper reports.

RHO is not the same denominator as useful-work efficiency. Section 2’s C_Q measures the resource cost of a validated task against a workload’s own baseline; RHO is a separate, internal resource-accounting semantic used for settlement between funded agent identities. A reader should

not infer cryptocurrency-style token economics from this paper’s commercial thesis: RHO has no defined market price, is not redeemable for a fixed unit of currency, and is not presented anywhere in this paper as evidence for or against the useful-work efficiency argument made in Sections 2 and 4.

Demonstrated settlement. A full three-operation economic strike — recall, write, and query — settled end-to-end on a public test network: per-model spending allowances were granted, spent to zero, and every replay attempt was reverted, with 500 RHO settled and independently recomputed from two different accounting paths.

Validation scope. Settlement is demonstrated on testnet; production-scale, multi-principal settlement and any market valuation of settled units are the next stage. No claim of live, production spending authority is made in this paper — identity and funding on a test network are not the same predicate as authorized production spend, per the separation stated above.

11. When Software Becomes Structural

As machine intelligence moves from producing information to controlling consequential systems — a financial transfer, an infrastructure change, a physical action — the software governing that transition stops being an implementation detail and becomes part of the system’s load-bearing structure, in the way a bridge’s structural members, not its paint, keep it standing. A structure that is correct on average is not sufficient when a specific, consequential transition is underway: the governing invariants must hold for *that* transition. This is the architectural reason reasoning and authority cannot be the same mechanism (Section 9), and it is the same reason Section 7 insists that a portable execution state does not carry its own authorization with it.

The primary consequence-active example: an aviation pre-flight authorization gate. A heavy-lift uncrewed aircraft program illustrates the requirement with unusual clarity: a physical machine with real authority to release cargo or return to service, continuous telemetry, explicit safety invariants, and consequences that cannot be undone once actuated. ExergyNet’s authorization gate for this program produces exactly three terminal states — release-eligible, hold, or incomplete — as an additional safety layer, deferring final authority to its own separate policy evaluation (**Figure 12**).

Across two independent validation runs — a deterministic rule-based simulator and, separately, a real reasoning model — the governed gate held false-release at 0 of 50 cases on a sealed synthetic test set, against a 20-34% false-release rate for the same cases evaluated on ungoverned raw telemetry (**Figure 8**).

Validation scope. This result is measured against a sealed, synthetic 50-case holdout; no real aircraft or sensor hardware was involved. A regulatory authorization exists for the related aircraft program permitting controlled testing and evaluation — a fact about that program’s regulatory status, not a certification of this architecture.

12. Applications

The architecture stands independently of any one vertical; removing this section leaves it intact.

Aviation (Section 11) is the primary demonstrated consequence-active application. **Clinical decision support** is a second, illustrative case study: a clinical assistant must preserve an authoritative patient-state record across sessions, must not promote unresolved evidence into that

record, and must keep model reasoning separate from any consequential action — properties this architecture is built to provide, presented here as an illustrative reference pattern rather than a deployed clinical integration.

Robotics and critical infrastructure, scientific and research agents, industrial automation, enterprise autonomous agents, financial and transactional agents, and secure machine-to-machine communications share the same underlying requirement — an identity-bound, evidence-backed, capability-scoped decision before a consequential action — and are architecturally applicable under the same protocol, without a bounded internal benchmark reported for each. An ExergyNet-maintained MCP (Model Context Protocol) server provides one such interface, exposing read-only protocol lookups and a local compute-cost estimator to compatible agent runtimes today; its write-settlement path is disabled pending a verified production contract migration, and a fresh, ordinary installation of the current package reported zero `npm audit` findings against its tested dependency tree. This interface is a way of reaching ExergyNet’s architecture from a compatible agent runtime — it is not the architecture itself, and this paper does not describe interface distribution status as evidence for or against any claim made elsewhere in this paper.

13. Validation Status and Research Frontier

Every component below is reported at its actual, current maturity stage: **Defined** (specified) -> **Implemented** (built) -> **Validated** (reproducibly exercised in a test envelope) -> **Pilot** / **Shadow** (running against real decisions, not yet the sole authority) -> **Production** (operating as the system of record).

Component	Maturity	Boundary
xLMP / Exergy Vault	Production	Published local node (VMN); integrity is distinct from correctness
Compact Index / Adaptive Routing	Validated -> Partial production	Performance is workload-dependent
Vanguard (model routing)	Production	Proposes actions; does not self-authorize
Authority validator (identity, capability, freshness, integrity)	Production	Live-validated against a real adversarial run
Policy / capability gate	Pilot	Evaluates and logs every real decision; not yet the sole execution authority
xISA capability taxonomy	Validated	Isolated research environment; not yet production-active
Temporal Authority	Implemented, tested	Not yet production-activated
RHO resource metrology	Pilot	Not monetary; GPU-cost term currently modeled, not measured
Omega	Implemented	No live spending authority currently granted
MMS settlement	Validated	Test network only

Component	Maturity	Boundary
AERIS (external evidence)	Validated	Test network; production settlement requires a further generation
Edge Witness (physical observation)	Production	Attests device signature, not physical truth
Execution-state mobility	Validated within a frozen same-model, cross-host envelope	Same model, same runtime, same quantization, one tested workload length, two x86-64 hosts; cross-model, cross-runtime, cross-ISA, and general-workload portability not demonstrated
Portable Intelligence Package	Defined / specification	Reference architecture; no production implementation claimed
Aviation pre-flight gate	Validated (synthetic holdout)	No real-aircraft validation yet reported
Physical-state observation (forward architecture)	Defined (proposed)	No implementation yet

Research frontier, stated plainly: production-wide authority enforcement across every live route; a physical-state observation primitive; production-scale zero-knowledge verification integrated into the evidence-query path; cross-model execution-state mapping; cross-runtime and cross-ISA/accelerator execution-state realization; replication of the recurrent-state causal structure reported in Section 5 across a third physical host and across other workload lengths; the mechanism behind the observed recurrent-state divergence itself, which remains unknown; further reduction of the S-state minimal structure; and the integration of the Portable Intelligence Package specification with VMN’s persistent project-state handoff path, which is not yet built. These are named as frontiers because they are not yet true, not because the architecture cannot eventually reach them, and none of them needs to be solved for the results already reported in Section 5 to be valid within their own tested envelope.

Proposed Comparative Evaluation Program

The useful-work measurement layer introduced in Section 2 motivates a standing comparative benchmark program against a common set of reference baselines:

- B_0 — full-context replay
- B_1 — provider-native caching
- B_2 — optimized RAG
- B_3 — persistent/provider memory
- B_4 — ExergyNet / xLMP
- B_5 — native optimization + ExergyNet

Section 4 reports results consistent with B_0 and B_2 (full-context replay and a tested RAG baseline) as T1, tested-envelope results. **No result is reported for B_1 , B_3 , or B_5 until an artifact exists in the claim ledger.** This program matters commercially because the relevant comparison

for an adopting organization is not xLMP against an intentionally expensive replay architecture — it is xLMP’s incremental value after that organization’s own already-optimized baseline, which is exactly what B_5 is designed to measure once run.

Conclusion

The AI industry is building the vehicles: increasingly capable, increasingly numerous, increasingly heterogeneous models, built by frontier labs, open-model communities, and specialized vendors alike. That buildout is now measured in tens of billions of dollars of quarterly infrastructure investment and, by its largest supplier’s own account, still accelerating. The models do not need to become interchangeable for that buildout to matter here — model plurality is precisely what makes shared infrastructure necessary, not optional.

The AI industry is building the vehicles. ExergyNet is building the roads required when those vehicles become numerous, heterogeneous, persistent, and autonomous.

More precisely: ExergyNet’s objective is a common machine infrastructure in which intelligence can persist, establish identity, preserve evidence, exercise bounded authority, account for resources, move its own execution state where that is possible, and interact with the physical world — without requiring those functions to reside inside any single model. Its contribution is a substrate that keeps knowledge, evidence, capability, authority, economy, and execution state as separate, independently verifiable properties, so that intelligence can be swapped, can fail, can move, or can be distrusted without the surrounding system losing what it knows, what it may do, or what it must account for. If every current frontier model were replaced tomorrow, the reason ExergyNet exists would be unchanged.

Design partners can provide a bounded, state-heavy workload for comparative measurement against their existing optimized baseline. **Benchmark the workload, not believe the claim.**

References

1. NVIDIA Corporation. *NVIDIA Announces Financial Results for Second Quarter Fiscal 2027*. Press release, August 26, 2026.
2. NVIDIA Corporation. *CFO Commentary, Second Quarter Fiscal 2027*. August 26, 2026.
3. NVIDIA Corporation. Second Quarter Fiscal 2027 earnings call remarks (J. Huang), August 26, 2026.
4. Huang, J. Public remarks on land, power, and shell capacity for AI infrastructure, August 2026.
5. Sreeramaneni, R. (Micron). *Evolving Memory Architectures for AI*. Hot Chips 2026, Stanford University, August 23, 2026.
6. U.S. Provisional Patent Application No. 64/134,973, filed 2026 (formal claims subject to prosecution).
7. Federal Aviation Administration. Exemption No. 26214, Docket FAA-2025-5731.

A full claim ledger, source registry, and revision history are maintained as separate internal diligence documents and are available on request.